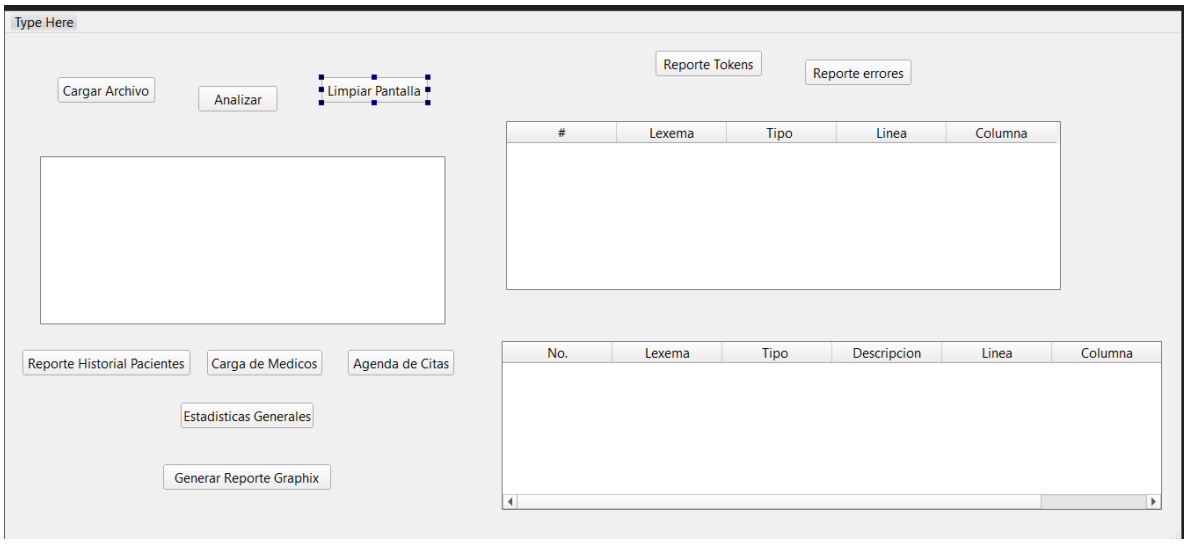
Manual Técnico

El sistema sigue un patrón de Arquitectura en Capas simple pero robusto, separando la interfaz de usuario de la lógica de procesamiento.

- Capa de Interfaz (GUI): Implementada en Qt (MainWindow), gestiona la interacción con el usuario y visualización de datos.



Aca se crearon desde el widget box, cada botón y cada tabla, respetando siempre los lineamientos necesarios.

El código se crea solo en mainwindow.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>MainWindow</class>
 <widget class="QMainWindow" name="MainWindow">
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>1338</width>
    <height>602</height>
   </rect>
  </property>
  <property name="windowTitle">
   <string>MainWindow</string>
  </property>
  <widget class="QWidget" name="centralwidget">
   <widget class="QPlainTextEdit" name="plainTextEdit">
    <property name="geometry">
     <rect>
      <x>40</x>
      <y>140</y>
      <width>461</width>
      <height>191</height>
     </rect>
    </property>
   </widget>
   <widget class="QPushButton" name="pushButton">
    <property name="geometry">
     <rect>
```

Ademas de ello se crearon 3 carpetas: Forms, Header Files, Source Files. El mainwindow esta presente en cada una de ellas.

- Capa de Lógica (Scanner): Motor de análisis léxico que procesa el flujo de caracteres.

En el mainwindow.h esta la inclusion de cabecera

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
```

También acá colocaremos los private slots:

```
private slots:
    void on_btnCargar_clicked();
    void on_pushButton_clicked();
    void on_btnReporteTokens_clicked();
    void on_ReporteHistorialPacientes_clicked();
    void on_btnCargaMedicos_clicked();
    void on_btnAgendaCitas_clicked();
    void on_btnEstaditicasG_clicked();
    void on_btnLimpiar_clicked();
    void on_btnReporteErrores_clicked();
    void on_btnGenerarReporteGrap_clicked();
    void llenarTablaTokens();
    void llenarTablaErrores();

    void on_tablaerrores_cellActivated(int row, int column);

    void on_tablaTokens_cellActivated(int row, int column);
```

Luego tenemos scanner.h que acá tenemos una de las cosas mas importantes, la enumeración de tipo de tokens

```
enum TipoToken {
    HOSPITAL, PACIENTES, MEDICOS, CITAS, DIAGNOSTICOS,
    PACIENTE_ELEMENTO, MEDICO_ELEMENTO, CITA_ELEMENTO, DIAGNOSTICO_ELEMENTO,
    ID_MEDICO, CADENA, NUMERO,
    LLAVE_A, LLAVE_C, CORCHETE_A, CORCHETE_C, DOS_PUNTOS, COMA,
    PUNTO_COMA,
    ERROR
};
```

Define los tipos de elementos léxicos que el scanner puede reconocer en la entrada.

Ejemplos:

HOSPITAL, PACIENTES → palabras clave del sistema.

ID_MEDICO, CADENA, NUMERO → tipos de datos.

LLAVE_A, CORCHETE_C → símbolos de estructura ({,], etc.).

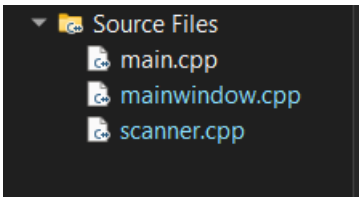
ERROR → token que representa un error léxico.

Tambien tenemos la estructura de datos:

```
struct Token {
    TipoToken tipo;
    QString lexema;
    int linea;
    int columna;
};

struct Diagnostico {
    QString paciente;
    QString condicion;
    QString medicamento;
    QString dosis;
};
```

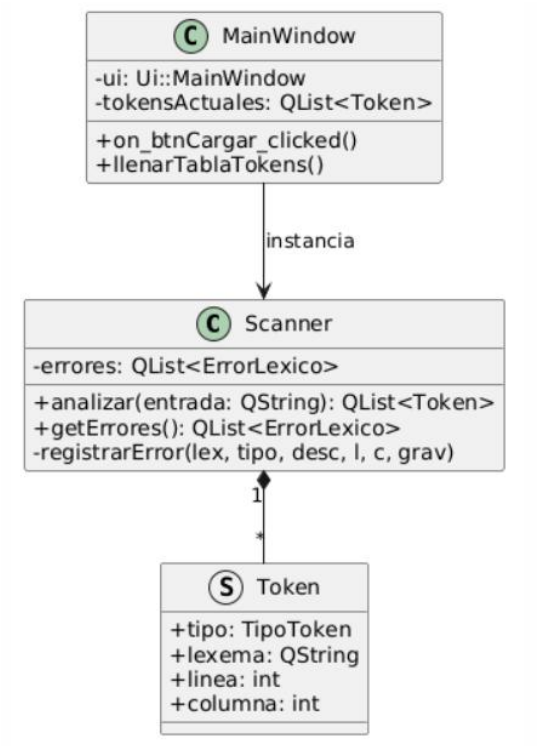
Luego en el source files tenemos:



Que acá se encuentran en la mayoría la funcionabilidad de los botones y de las funciones de los lexemas, además el html que se genera al momento de ingresar o darle click.

- Gestor de Errores: Registra incidencias sin detener el flujo del análisis, permitiendo un reporte completo al finalizar.

- Diagrama de clases:



- Diagrama de AFD:

En general esto es lo que hace el programa

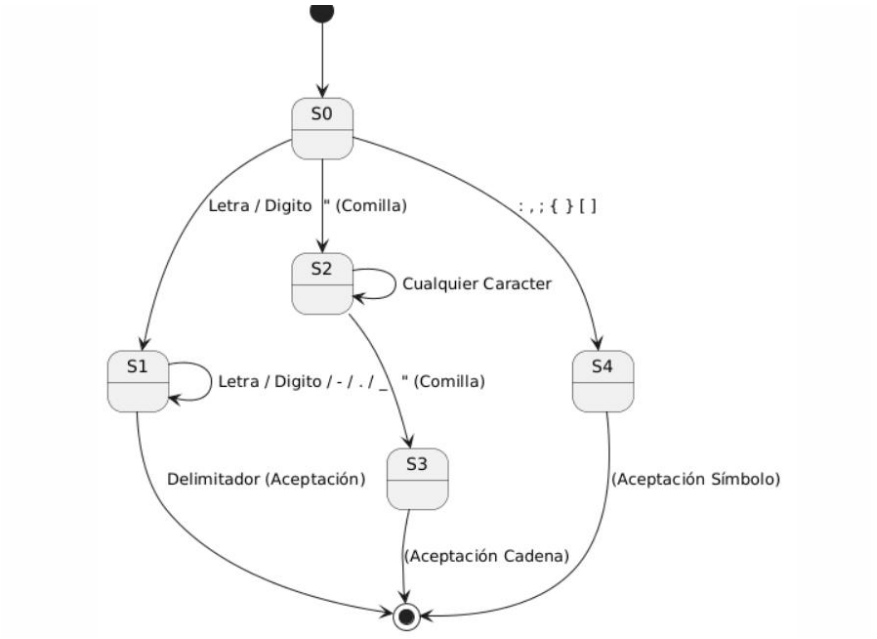


TABLA DE TRANSICIONES:

Estado	**Letra/Dígito**	**Comilla (")**	**Símbolos**	**Espacio/Enter**
S0	S1	S2	S4 (Aceptación)	S0
S1	S1	Error	Aceptación (i--)	Aceptación
S2	S2	S3	S2	S2
S3	Error	Error	Aceptación	

- Justificación de Diseño:

Uso de C++17 y Qt Framework: Se seleccionó Qt por su robustez en el manejo de interfaces y la gestión eficiente de memorias con QList.

Cumplimiento de Restricciones: El scanner fue programado carácter por carácter. Se omitió totalmente el uso de std::regex para cumplir con la restricción de tokenización manual.

Validación Lógica Post-Análisis: Para las fechas y horas, el scanner captura el lexema completo y luego valida su contenido (rangos de meses de 1-12, días 1-31) mediante funciones de conversión nativas, garantizando precisión sin complicar el AFD.

Retroceso de Puntero (i--): Se implementó la técnica de retroceso para que los caracteres delimitadores (como las llaves) no sean consumidos por el estado anterior, permitiendo que sean procesados como tokens individuales.